



1. Fase de Infraestructura (VM Ubuntu)

Antes de desplegar código, debemos preparar el entorno para evitar que el sistema colapse por falta de memoria.

- **Implementación de SWAP:** Crear un archivo de intercambio de **2GB** en Ubuntu para dar aire al sistema cuando los procesos de Python y .NET coincidan en uso de memoria.
 - **Orquestación con Docker Compose:** Configurar un archivo `docker-compose.yml` que gestione tres contenedores: la WebApp (.NET), el AI-Service (Python) y la base de datos SQL.
 - **Red Interna:** Definir una `bridge network` en Docker para que los contenedores se comuniquen por nombres de servicio (ej: `http://ai-service:8000`), manteniendo los puertos del RAG cerrados al exterior.
-

2. Fase del Microservicio RAG (Python + FastAPI)

Transformaremos tus scripts actuales (`chat.py` y `rag_arandano.py`) en una API robusta.

A. Estructura de Colecciones en ChromaDB

- **Colección `papers_expertos`** : Solo lectura en producción. Se actualiza mediante la carga de la DB pre-procesada en tu PC (Dell Inspiron).
- **Colección `bitacoras_usuario`** : Lectura y escritura. Almacena las observaciones enviadas desde el monolito.

B. Endpoints de la API

- **`POST /chat`** : Recibe la pregunta, el contexto IoT y decide qué modelo de Gemini usar (Lite o Flash).
- **`POST /ingest-logs`** : Recibe nuevas bitácoras desde .NET y las vectoriza en la colección correspondiente.
- **`POST /update-papers`** : Recibe un archivo ZIP, reemplaza la colección de papers y refresca el cliente de Chroma sin reiniciar el contenedor.

C. Seguridad

- **X-API-KEY:** Implementación de un Middleware que valide un token secreto en cada petición, compartido únicamente con el monolito de .NET.
-

3. Fase del Monolito (.NET Core 8)

Aquí se gestiona la lógica de negocio, los datos del usuario y la interfaz.

A. Cambios en el Dominio y Datos

- **Entidad `observation.cs`** : Agregar propiedad `bool IsVectorized` para controlar qué registros ya están en el RAG.
- **Base de Datos SQL:** Asegurar que cada observación tenga un ID único que se pasará a Chroma como metadato para evitar duplicados.

B. Capa de Aplicación e Infraestructura

- **`IRagService`** : Nueva interfaz para centralizar las llamadas al microservicio de IA.
- **Selector de Contexto IoT:** Lógica para calcular promedios de sensores o extraer los últimos 20 registros antes de enviarlos al chat.
- **Background Service (Worker):** Un servicio de segundo plano que se active cada madrugada (ej. 2:00 AM) para enviar las bitácoras pendientes al RAG.

C. Presentación (UI)

- **Interfaz de Chat:** Nueva vista en `/Admin/Chat` con un componente de chat interactivo.
 - **Gestor de Base de Datos:** Opción en el panel de administración para subir el archivo ZIP con la nueva versión de la base de datos vectorial de papers.
-

4. Fase de Gestión de Datos (Flujo de Trabajo Externo)

Dado que no procesaremos embeddings de PDFs en el servidor de 2GB para ahorrar recursos:

- **Local Processing:** Usarás tu laptop (Dell Inspiron, 16GB RAM) para ejecutar `setup_database.py` con los nuevos PDFs.
- **Deployment:** Generarás el ZIP de la carpeta `chroma_db` (solo la colección de papers) y lo cargarás a través del panel administrativo.